

Contents

Módulo 5 - Práctica: gestión de errores y configuración avanzada	2
Recorridos de entorno	2
Consola M5	2
IntelliJ IDEA M5	2
Eclipse M5	3
VS Code M5	3
Práctica 5.1 - Excepciones personalizadas	3
Paso 1 - Repasar el estado actual	3
Paso 2 - Crear el paquete exception	3
Paso 3 - Crear la clase base AplicacionException	3
Paso 4 - Crear RecursoNoEncontradoException	4
Paso 5 - Crear RecursoDuplicadoException	4
Paso 6 - Crear OperacionNoPermitidaException	5
Paso 7 - Crear ErrorTecnicoException	5
Paso 8 - Refactorizar el AlumnoService	5
Paso 9 - Refactorizar el CursoService	7
Paso 10 - Escribir tests para las excepciones	8
Paso 11 - Ejecutar los tests	9
Paso 12 - Errores comunes del ejercicio	9
Paso 13 - Reto resuelto — Excepción para validación de negocio	9
Resultado esperado global	10
Resumen técnico	10
Conclusión	11
Práctica 5.2 - Respuestas de error estándar	11
Paso 1 - Repasar el estado actual	11
Paso 2 - Crear el paquete dto.error	11
Paso 3 - Crear la clase ValidationError	11
Paso 4 - Crear la clase ErrorResponse	12
Paso 5 - Refactorizar el GlobalExceptionHandler	13
Paso 6 - Añadir manejadores para errores de parámetros	16
Paso 7 - Arrancar y probar errores	17
Paso 8 - Verificar el error 404	17
Paso 9 - Verificar el error de validación	17
Paso 10 - Escribir tests para las respuestas de error	18
Paso 11 - Errores comunes del ejercicio	19
Paso 12 - Reto resuelto — Añadir un campo traceId a las respuestas de error	20
Resultado esperado global	21
Resumen técnico	21
Conclusión	21
Práctica 5.3 - Manejo global de excepciones	22
Paso 2 - Crear el ValidationExceptionHandler	22
Paso 3 - Crear el BusinessExceptionHandler	24
Paso 4 - Crear el GenericExceptionHandler	25
Paso 5 - Eliminar el GlobalExceptionHandler antiguo	27
Paso 6 - Arrancar y probar el orden	27
Paso 7 - Aplicar un manejador a un paquete concreto	28
Paso 8 - Escribir tests para el orden	28
Paso 9 - Verificar el traceId	29
Paso 10 - Añadir un manejador para errores de tipo	30
Paso 11 - Errores comunes del ejercicio	30
Paso 12 - Reto resuelto — Manejador específico para ficheros	30
Resultado esperado global	31

Conclusión	32
Práctica 5.4 - Configuración básica de perfiles y properties	32
Paso 1 - Repasar el estado actual	32
Paso 2 - Reorganizar application.properties	32
Paso 3 - Crear application-dev.properties	33
Paso 4 - Crear application-test.properties	33
Paso 5 - Crear application-prod.properties	34
Paso 6 - Activar el perfil dev	34
Paso 7 - Probar con el perfil prod	35
Paso 8 - Crear la clase AppProperties	35
Paso 9 - Añadir un bean condicional por perfil	36
Paso 10 - Arrancar con distintos perfiles	37
Paso 11 - Errores comunes del ejercicio	37
Paso 12 - Reto resuelto — Test con perfil específico	38
Resultado esperado global	38
Resumen técnico	39
Práctica 5.5 - Configuración de CORS para consumo desde front-end	39
Paso 1 - Repasar el estado actual	39
Paso 2 - Simular una petición desde otro origen	40
Paso 3 - Crear la clase CorsConfig	40
Paso 4 - Arrancar y probar con curl	40
Paso 5 - Probar con un origen no permitido	41
Paso 6 - Simular un preflight	41
Paso 7 - Probar con un método no permitido	41
Paso 8 - Configurar CORS con perfiles	42
Paso 9 - Manejar CORS en errores	42
Paso 10 - Errores comunes del ejercicio	43
Paso 11 - Reto resuelto — Probar CORS con un front-end real	43
Resultado esperado global	44
Resumen técnico	45
Conclusión	45

Módulo 5 - Práctica: gestión de errores y configuración avanzada

Regla de ejecución: cada paso debe dejar un observable verificable. Todo cambio temporal de perfil, CORS o diagnóstico se restaura antes de cerrar el paso o se consolida explícitamente en el estado final.

Continuidad con M4-R1: se conservan JPA/Hibernate, los 23 tests acumulativos y la separación AlumnoRequestDT0 / AlumnoResponseDT0. Cuando la fuente usa nombres anteriores, el ejercicio se adapta al contrato acumulativo sin perder el concepto que se enseña.

Recorridos de entorno

Consola M5

Ejecuta `./mvnw` en Linux/macOS o `mvnw.cmd` en Windows desde M5/proyecto. Usa `curl -i` para observar códigos, cabeceras CORS y cuerpos JSON de error.

IntelliJ IDEA M5

Importa M5/proyecto/pom.xml como proyecto Maven, selecciona Java 17 y ejecuta MiProyectoApplication o los tests con los perfiles indicados en cada paso.

Eclipse M5

Importa M5/proyecto como Existing Maven Project, selecciona Java 17 y usa Run Configurations para cambiar spring.profiles.active sin modificar el snapshot permanente.

VS Code M5

Abre M5/proyecto, conserva Java 17 y ejecuta siempre el Maven Wrapper incluido. Para CORS, observa también las cabeceras HTTP en la terminal o en el navegador.

Práctica 5.1 - Excepciones personalizadas

Contexto del ejercicio: Vamos a crear una jerarquía de excepciones personalizadas para el proyecto, refactorizar el AlumnoService para usarlas, y escribir tests que verifiquen que se lanzan correctamente.

Requisitos previos: Tener el proyecto mi-proyecto con el AlumnoService del Módulo 4.

Paso 1 - Repasar el estado actual

Abre el AlumnoService y observa cómo se lanzan las excepciones actualmente:

```
if (repositorio.existsByDni(request.getDni())) {  
    throw new NegocioException("Ya existe un alumno con el DNI " + request.getDni());  
}
```

Tienes una excepción NegocioException genérica. Funciona, pero no distingue entre tipos de error.

Pregunta de reflexión: ¿Qué problemas tiene usar una sola excepción genérica para todos los errores?

Paso 2 - Crear el paquete exception

Crea el paquete es.mecd.demo.miproyecto.common.exception (o es.mecd.demo.miproyecto.exception si mantienes la estructura por capas).

Pregunta de reflexión: ¿Por qué conviene tener un paquete específico para las excepciones?

Paso 3 - Crear la clase base AplicacionException

Crea la clase base:

```
package es.mecd.demo.miproyecto.common.exception;  
  
public abstract class AplicacionException extends RuntimeException {  
    private final String codigo;  
  
    protected AplicacionException(String codigo, String mensaje) {  
        super(mensaje);  
        this.codigo = codigo;  
    }  
  
    protected AplicacionException(String codigo, String mensaje, Throwable causa) {  
        super(mensaje, causa);  
        this.codigo = codigo;  
    }  
}
```

```
public String getCodigo() {  
    return codigo;  
}  
}
```

abstract porque no se instancia directamente.

codigo es el identificador interno del error. Dos constructores: uno con mensaje, otro con mensaje y causa.

Pregunta de reflexión: ¿Por qué la clase base es abstract? ¿Qué pasaría si no lo fuera?

Paso 4 - Crear RecursoNoEncontradoException

Crea la excepción para recursos no encontrados:

```
package es.mecd.demo.miproyecto.common.exception;  
  
public class RecursoNoEncontradoException extends AplicacionException {  
  
    private final String recurso;  
    private final Object id;  
  
    public RecursoNoEncontradoException(String recurso, Object id) {  
        super("RECURSO_NO_ENCONTRADO",  
            String.format("No se encontró %s con id %s", recurso, id));  
        this.recurso = recurso;  
        this.id = id;  
    }  
  
    public String getRecurso() { return recurso; }  
    public Object getId() { return id; }  
}
```

`super(...)` llama al constructor de la clase base con el código y el mensaje.

`String.format` construye el mensaje con los datos concretos.

Getters para que el manejador pueda acceder a los datos.

Pregunta de reflexión: ¿Qué código de error usa esta excepción? ¿Por qué es importante que sea estable?

Paso 5 - Crear RecursoDuplicadoException

Crea la excepción para recursos duplicados:

```
package es.mecd.demo.miproyecto.common.exception;  
  
public class RecursoDuplicadoException extends AplicacionException {  
    private final String recurso;  
    private final String campo;  
    private final Object valor;  
  
    public RecursoDuplicadoException(String recurso, String campo, Object valor) {  
        super("RECURSO_DUPLICADO",  
            String.format("Ya existe %s con %s = %s", recurso, campo, valor));  
        this.recurso = recurso;  
        this.campo = campo;  
        this.valor = valor;  
    }  
}
```

```

    }

    public String getRecurso() { return recurso; }
    public String getCampo() { return campo; }
    public Object getValor() { return valor; }
}

```

Pregunta de reflexión: ¿Qué información adicional lleva esta excepción que no lleva RecursoNoEncontradoException?

Paso 6 - Crear OperacionNoPermitidaException

Crea la excepción para operaciones no permitidas:

```

package es.mecd.demo.miproyecto.common.exception;

public class OperacionNoPermitidaException extends AplicacionException {

    public OperacionNoPermitidaException(String mensaje) {
        super("OPERACION_NO_PERMITIDA", mensaje);
    }
}

```

Pregunta de reflexión: ¿En qué casos usarías esta excepción? ¿Qué código HTTP se traduciría?

Paso 7 - Crear ErrorTecnicoException

Crea la excepción para errores técnicos:

```

package es.mecd.demo.miproyecto.common.exception;

public class ErrorTecnicoException extends AplicacionException {

    public ErrorTecnicoException(String mensaje, Throwable causa) {
        super("ERROR_TECNICO", mensaje, causa);
    }
}

```

Pregunta de reflexión: ¿Por qué esta excepción tiene un constructor que recibe una causa?

Paso 8 - Refactorizar el AlumnoService

Modifica el AlumnoService para usar las nuevas excepciones:

```

@Service
public class AlumnoService {

    private final AlumnoRepository repositorio;
    private final CursoRepository cursoRepository;

    public AlumnoService(
        AlumnoRepository repositorio,
        CursoRepository cursoRepository) {
        this.repositorio = repositorio;
        this.cursoRepository = cursoRepository;
    }

    @Transactional(readOnly = true)

```

```
public Optional<AlumnoDTO> consultar(Long id) {
    return repositorio.findById(id).map(this::toDTO);
}

@Transactional
public AlumnoDTO crear(AlumnoRequestDTO request) {
    if (repositorio.existsByDni(request.getDni())) {
        throw new RecursoDuplicadoException("Alumno", "dni", request.getDni());
    }
    Curso curso = cursoRepository.findById(request.getCurso())
        .orElseThrow(() -> new RecursoNoEncontradoException("Curso", request.getCurso()));
    Alumno entidad = new Alumno(
        request.getNombre(),
        request.getApellidos(),
        request.getDni(),
        request.getFechaNacimiento()
    );
    entidad.setCurso(curso);
    return toDTO(repositorio.save(entidad));
}

@Transactional
public Optional<AlumnoDTO> actualizar(Long id, AlumnoRequestDTO request) {
    return repositorio.findById(id).map(entidad -> {
        if (!entidad.getDni().equals(request.getDni())
            && repositorio.existsByDni(request.getDni())) {
            throw new RecursoDuplicadoException("Alumno", "dni", request.getDni());
        }
        entidad.setNombre(request.getNombre());
        entidad.setApellidos(request.getApellidos());
        entidad.setDni(request.getDni());
        entidad.setFechaNacimiento(request.getFechaNacimiento());
        return toDTO(repositorio.save(entidad));
    });
}

@Transactional
public void eliminar(Long id) {
    Alumno alumno = repositorio.findById(id)
        .orElseThrow(() -> new RecursoNoEncontradoException("Alumno", id));
    repositorio.delete(alumno);
}

private AlumnoDTO toDTO(Alumno entidad) {
    AlumnoDTO dto = new AlumnoDTO();
    dto.setIdentificador(
        entidad.getId() != null
            ? entidad.getId().toString()
            : null);
    dto.setNombre(entidad.getNombre());
    dto.setApellidos(entidad.getApellidos());
    dto.setDni(entidad.getDni());
    dto.setFechaNacimiento(entidad.getFechaNacimiento());
    dto.setCurso(entidad.getCurso() != null ? entidad.getCurso().getNombre() : null);
    return dto;
}
```

```
}
```

Cambios principales:

- RecursoDuplicadoException en lugar de NegocioException para el DNI duplicado.
- RecursoNoEncontradoException cuando no se encuentra el curso.
- RecursoNoEncontradoException cuando no se encuentra el alumno para eliminar.
- Optional.map en actualizar: si no existe, devuelve Optional.empty().

Pregunta de reflexión: ¿Por qué actualizar devuelve Optional en lugar de lanzar excepción si no existe?

Paso 9 - Refactorizar el CursoService

Modifica el CursoService de forma similar:

```
@Service
public class CursoService {

    private final CursoRepository repositorio;

    public CursoService(CursoRepository repositorio) {
        this.repositorio = repositorio;
    }

    @Transactional
    public CursoDTO crear(String nombre) {
        if (repositorio.existsByNombre(nombre)) {
            throw new RecursoDuplicadoException("Curso", "nombre", nombre);
        }
        return toDTO(repositorio.save(new Curso(nombre)));
    }

    @Transactional
    public void eliminar(Long id) {
        Curso curso = repositorio.findById(id)
            .orElseThrow(() -> new RecursoNoEncontradoException("Curso", id));
        if (!curso.getAlumnos().isEmpty()) {
            throw new OperacionNoPermitidaException(
                "No se puede eliminar el curso " + id
                + ": tiene " + curso.getAlumnos().size() + " alumnos");
        }
        repositorio.delete(curso);
    }

    private CursoDTO toDTO(Curso curso) {
        CursoDTO dto = new CursoDTO();
        dto.setIdentificador(curso.getId() != null ? curso.getId().toString() : null);
        dto.setNombre(curso.getNombre());
        dto.setNumeroAlumnos(curso.getAlumnos().size());
        return dto;
    }
}
```

Pregunta de reflexión: ¿Qué excepción se lanza si intentas eliminar un curso con alumnos? ¿Qué código HTTP se traduciría?

Paso 10 - Escribir tests para las excepciones

Añade tests al AlumnoServiceTest:

```
@Test
void crear_debeLanzarRecursoDuplicado_cuandoDniExiste() {
    when(repositorio.existsByDni("12345678A")).thenReturn(true);
    AlumnoRequestDTO request = new AlumnoRequestDTO();
    request.setDni("12345678A");

    RecursoDuplicadoException ex = assertThrows(
        RecursoDuplicadoException.class,
        () -> service.crear(request)
    );

    assertEquals("RECURSO_DUPLICADO", ex.getCodigo());
    assertEquals("dni", ex.getCampo());
    assertEquals("12345678A", ex.getValor());
}

@Test
void crear_debeLanzarRecursoNoEncontrado_cuandoCursoNoExiste() {
    when(repositorio.existsByDni("12345678A")).thenReturn(false);
    when(cursoRepository.findByNombre("Curso Inexistente"))
        .thenReturn(Optional.empty());
    AlumnoRequestDTO request = new AlumnoRequestDTO();
    request.setDni("12345678A");
    request.setCurso("Curso Inexistente");

    RecursoNoEncontradoException ex = assertThrows(
        RecursoNoEncontradoException.class,
        () -> service.crear(request)
    );
    assertEquals("RECURSO_NO_ENCONTRADO", ex.getCodigo());
    assertEquals("Curso", ex.getRecurso());
    assertEquals("Curso Inexistente", ex.getId());
}

@Test
void crear_noDebeLanzarExcepcion_cuandoDatosValidos() {
    when(repositorio.existsByDni("12345678A")).thenReturn(false);
    when(cursoRepository.findByNombre("5º Primaria"))
        .thenReturn(Optional.of(new Curso("5º Primaria")));
    when(repositorio.save(any())).thenAnswer(inv -> inv.getArgument(0));
    AlumnoRequestDTO request = new AlumnoRequestDTO();
    request.setNombre("Ana");
    request.setApellidos("García");
    request.setDni("12345678A");
    request.setCurso("5º Primaria");

    assertDoesNotThrow(() -> service.crear(request));
}
```

Los tres tests cubren:

- Que se lanza RecursoDuplicadoException con los datos correctos.
- Que se lanza RecursoNoEncontradoException con los datos correctos.
- Que no se lanza excepción cuando los datos son válidos.

Pregunta de reflexión: ¿Por qué el tercer test es importante? ¿Qué error detecta?

Paso 11 - Ejecutar los tests

Ejecuta los tests:

```
./mvnw test -Dtest=AlumnoServiceTest
```

Verás algo como:

```
Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Pregunta de reflexión: ¿Qué ventaja tiene que los tests verifiquen tanto que se lanza como que no se lanza la excepción?

Paso 12 - Errores comunes del ejercicio

Error	Causa	Solución
No se compila ClassCastException en el manejador	Falta el import de la excepción Se espera un tipo y llega otro	Añadirlo Verificar el tipo
El mensaje no se construye	Falta super(...) en el constructor	Añadirlo
La causa no se guarda assertThrows no detecta	Falta el constructor con causa La excepción lanzada es distinta	Añadirlo Verificar el tipo
El test pasa sin excepción La excepción se captura sin relanzar	No se ha configurado el mock try-catch mal usado	Añadir when(...) Relanzar o no capturar
El código de error cambia	Se ha escrito como mensaje	Usar un código estable

Paso 13 - Reto resuelto — Excepción para validación de negocio

Reto: Crear una excepción ValidacionNegocioException que se lance cuando una regla de negocio no se cumple. Debe llevar el campo y el motivo.

Solución paso a paso:

Subpaso 1. Crear la excepción:

```
package es.mecd.demo.miproyecto.common.exception;
```

```
public class ValidacionNegocioException extends AplicacionException {
```

```
    private final String campo;
    private final String motivo;
```

```
    public ValidacionNegocioException(String campo, String motivo) {
        super("VALIDACION_NEGOCIO",
            String.format("El campo %s no cumple la regla: %s", campo, motivo));
        this.campo = campo;
        this.motivo = motivo;
    }
```

```
    public String getCampo() { return campo; }
    public String getMotivo() { return motivo; }
}
```

Subpaso 2. Añadir una regla de negocio en el servicio. Por ejemplo, que el nombre no contenga números:

```
private void validarNombre(String nombre) {
    if (nombre != null && nombre.matches(".*\\d.*")) {
        throw new ValidacionNegocioException("nombre", "no puede contener números");
    }
}
```

Subpaso 3. Llamar a la validación en crear:

```
@Transactional
public AlumnoDTO crear(AlumnoRequestDTO request) {
    validarNombre(request.getNombre());
    if (repositorio.existsByDni(request.getDni())) {
        throw new RecursoDuplicadoException("Alumno", "dni", request.getDni());
    }
    // ...
}
```

Subpaso 4. Escribir el test:

```
@Test
void crear_debeLanzarValidacionNegocio_cuandoNombreTieneNumeros() {
    AlumnoRequestDTO request = new AlumnoRequestDTO();
    request.setNombre("Ana123");
    request.setDni("12345678A");

    ValidacionNegocioException ex = assertThrows(
        ValidacionNegocioException.class,
        () -> service.crear(request)
    );

    assertEquals("nombre", ex.getCampo());
    assertEquals("no puede contener números", ex.getMotivo());
}
```

Pregunta de reflexión: ¿Qué diferencia hay entre ValidacionNegocioException y RecursoDuplicadoException?

Resultado esperado global

Al final del ejercicio, deberías tener: - Una clase base AplicacionException. - Cuatro excepciones específicas: RecursoNoEncontradoException, RecursoDuplicadoException, OperacionNoPermitidaException, ErrorTecnicoException. - Un AlumnoService y un CursoService que lanzan las excepciones correctas. - Tests que verifican que se lanzan y que no se lanzan. - (Opcional) Una ValidacionNegocioException.

Resumen técnico

El ejercicio ha demostrado:

- Clase base AplicacionException: agrupa todas las excepciones del proyecto.
- Excepciones específicas: cada tipo de error tiene su clase.
- Código de error interno: RECURSO_NO_ENCONTRADO, RECURSO_DUPLICADO, OPERACION_NO_PERMITIDA, ERROR_TECNICO.
- Datos adicionales: recurso, campo, valor, ID, causa.
- Dónde lanzar: desde el servicio.
- Envolver excepciones técnicas: para darles significado de negocio.
- Tests: verificar que se lanzan y que no se lanzan.

- Buenas prácticas: clase base, código estable, datos adicionales, no exponer detalles técnicos.

Conclusión

En este punto 5.1 hemos creado una jerarquía de excepciones personalizadas para el proyecto:

- Clase base `AplicacionException`: abstracta, con código de error.
- Excepciones específicas: `RecursoNoEncontradoException`, `RecursoDuplicadoException`, `OperacionNoPermitidaException`, `ErrorTecnicoException`.
- Datos adicionales: recurso, campo, valor, ID, causa.
- Lanzar desde el servicio: con contexto.
- Envolver excepciones técnicas: para darles significado.
- Tests: verificar que se lanzan y que no se lanzan.

En la práctica, hemos refactorizado el `AlumnoService` y el `CursoService` para usar las nuevas excepciones, y hemos escrito tests que verifican su comportamiento. La idea clave: las excepciones personalizadas permiten que el manejador global sepa qué tipo de error ha ocurrido y qué código HTTP devolver. Sin ellas, todos los errores son iguales y el cliente recibe respuestas genéricas.

En el siguiente punto, 5.2 - Respuestas de error estándar, profundizaremos en cómo estructurar las respuestas de error para que sean consistentes, informativas y útiles para el cliente.

Fin del Punto 5.1.

Práctica 5.2 - Respuestas de error estándar

Contexto del ejercicio: Vamos a crear los DTOs `ErrorResponse` y `ValidationError`, y a refactorizar el `GlobalExceptionHandler` para que devuelva respuestas de error con la estructura estándar. Veremos cómo se comporta con distintos tipos de error.

Requisitos previos: Tener el proyecto mi-proyecto con las excepciones personalizadas del punto 5.1 y el `GlobalExceptionHandler` básico.

Paso 1 - Repasar el estado actual

Abre el `GlobalExceptionHandler` y observa cómo se construyen las respuestas de error actualmente. Probablemente devuelve `Map<String, Object>` con campos sueltos. Vamos a refactorizarlo para usar DTOs tipados.

Pregunta de reflexión: ¿Qué problemas tiene devolver un `Map<String, Object>` en lugar de un DTO tipado?

Paso 2 - Crear el paquete `dto.error`

Crea el paquete `es.mecd.demo.miproyecto.common.dto.error` (o `es.mecd.demo.miproyecto.dto.error` si mantienes la estructura por capas).

Pregunta de reflexión: ¿Por qué conviene tener un paquete específico para los DTOs de error?

Paso 3 - Crear la clase `ValidationError`

Crea la clase `ValidationError`:

```
package es.mecd.demo.miproyecto.common.dto.error;

import com.fasterxml.jackson.annotation.JsonInclude;

@JsonInclude(JsonInclude.Include.NON_NULL)
```

```

public class ValidationError {

    private String field;
    private Object rejectedValue;
    private String message;

    public ValidationError() {
    }

    public ValidationError(String field, Object rejectedValue, String message) {
        this.field = field;
        this.rejectedValue = rejectedValue;
        this.message = message;
    }

    public String getField() { return field; }
    public void setField(String field) { this.field = field; }

    public Object getRejectedValue() { return rejectedValue; }
    public void setRejectedValue(Object rejectedValue) {
        this.rejectedValue = rejectedValue;
    }

    public String getMessage() { return message; }
    public void setMessage(String message) { this.message = message; }
}

```

@JsonInclude(NON_NULL) omite el campo rejectedValue si es null. En el caso de una validación de campo obligatorio, el valor rechazado es null; no tiene sentido mostrarlo.

Pregunta de reflexión: ¿Por qué el campo rejectedValue es de tipo Object y no String?

Paso 4 - Crear la clase ErrorResponse

Crea la clase ErrorResponse:

```

package es.mecd.demo.miproyecto.common.dto.error;

import com.fasterxml.jackson.annotation.JsonInclude;

import java.util.List;
import java.util.Map;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponse {

    private String timestamp;
    private int status;
    private String codigo;
    private String mensaje;
    private String path;
    private Map<String, Object> detalles;
    private List<ValidationError> errors;
    public ErrorResponse() {
    }

    public ErrorResponse(String timestamp, int status, String codigo,
        String mensaje, String path) {
    }
}

```

```

    this.timestamp = timestamp;
    this.status = status;
    this.codigo = codigo;
    this.mensaje = mensaje;
    this.path = path;
}

public String getTimestamp() { return timestamp; }
public void setTimestamp(String timestamp) { this.timestamp = timestamp; }

public int getStatus() { return status; }
public void setStatus(int status) { this.status = status; }

public String getCodigo() { return codigo; }
public void setCodigo(String codigo) { this.codigo = codigo; }

public String getMensaje() { return mensaje; }
public void setMensaje(String mensaje) { this.mensaje = mensaje; }

public String getPath() { return path; }
public void setPath(String path) { this.path = path; }

public Map<String, Object> getDetalles() { return detalles; }
public void setDetalles(Map<String, Object> detalles) { this.detalles = detalles; }

public List<ValidationError> getErrors() { return errors; }
public void setErrors(List<ValidationError> errors) { this.errors = errors; }
}

```

@JsonInclude(NON_NULL) omite los campos nulos. Así, un error sin detalles no incluye el campo en el JSON.

Pregunta de reflexión: ¿Qué campos aparecen siempre en el JSON y cuáles solo cuando tienen valor?

Paso 5 - Refactorizar el GlobalExceptionHandler

Modifica el GlobalExceptionHandler para usar los nuevos DTOs:

```

package es.mecd.demo.miproyecto.common.exception;

import es.mecd.demo.miproyecto.common.dto.error.ErrorResponse;
import es.mecd.demo.miproyecto.common.dto.error.ValidationError;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;
import java.util.List;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RecursoNoEncontradoException.class)

```

```
public ResponseEntity<ErrorResponse> handleNoEncontrado(
    RecursoNoEncontradoException ex,
    HttpServletRequest request) {
    return buildErrorResponse(
        HttpStatus.NOT_FOUND,
        ex.getCodigo(),
        ex.getMessage(),
        request,
        Map.of("recurso", ex.getRecurso(), "id", ex.getId())
    );
}

@ExceptionHandler(RecursoDuplicadoException.class)
public ResponseEntity<ErrorResponse> handleDuplicado(
    RecursoDuplicadoException ex,
    HttpServletRequest request) {
    return buildErrorResponse(
        HttpStatus.CONFLICT,
        ex.getCodigo(),
        ex.getMessage(),
        request,
        Map.of(
            "recurso", ex.getRecurso(),
            "campo", ex.getCampo(),
            "valor", ex.getValor()
        )
    );
}

@ExceptionHandler(OperacionNoPermitidaException.class)
public ResponseEntity<ErrorResponse> handleOperacionNoPermitida(
    OperacionNoPermitidaException ex,
    HttpServletRequest request) {
    return buildErrorResponse(
        HttpStatus.CONFLICT,
        ex.getCodigo(),
        ex.getMessage(),
        request,
        null
    );
}

@ExceptionHandler(ErrorTecnicoException.class)
public ResponseEntity<ErrorResponse> handleErrorTecnico(
    ErrorTecnicoException ex,
    HttpServletRequest request) {
    return buildErrorResponse(
        HttpStatus.INTERNAL_SERVER_ERROR,
        ex.getCodigo(),
        "Error interno del servidor",
        request,
        null
    );
}

@ExceptionHandler(MethodArgumentNotValidException.class)
```

```

public ResponseEntity<ErrorResponse> handleValidacion(
    MethodArgumentNotValidException ex,
    HttpServletRequest request) {
    List<ValidationError> errores = ex.getBindingResult()
        .getFieldErrors()
        .stream()
        .map(fe -> new ValidationError(
            fe.getField(),
            fe.getRejectedValue(),
            fe.getDefaultMessage()))
        .toList();

    ErrorResponse error = new ErrorResponse();
    error.setTimestamp(Instant.now().toString());
    error.setStatus(HttpStatus.BAD_REQUEST.value());
    error.setCodigo("VALIDACION");
    error.setMensaje("La petición tiene errores de validación");
    error.setPath(request.getRequestURI());
    error.setErrors(errores);
    return ResponseEntity.badRequest().body(error);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleGenerico(
    Exception ex,
    HttpServletRequest request) {
    return buildErrorResponse(
        HttpStatus.INTERNAL_SERVER_ERROR,
        "ERROR_INTERNO",
        "Error interno del servidor",
        request,
        null
    );
}

private ResponseEntity<ErrorResponse> buildErrorResponse(
    HttpStatus status,
    String codigo,
    String mensaje,
    HttpServletRequest request,
    Map<String, Object> detalles) {
    ErrorResponse error = new ErrorResponse();
    error.setTimestamp(Instant.now().toString());
    error.setStatus(status.value());
    error.setCodigo(codigo);
    error.setMensaje(mensaje);
    error.setPath(request.getRequestURI());
    error.setDetalles(detalles);
    return ResponseEntity.status(status).body(error);
}
}

```

Observa los cambios: - Cada manejador devuelve ResponseEntity, no Map. - El método buildErrorResponse centraliza la construcción. - MethodArgumentNotValidException tiene su propio manejador porque necesita construir la lista de ValidationError. - ErrorTecnicoException devuelve un mensaje genérico ("Error interno del servidor"), no el mensaje original. Los detalles técnicos no se exponen al cliente.

Pregunta de reflexión: ¿Por qué `ErrorTecnicoException` no expone el mensaje original?

Paso 6 - Añadir manejadores para errores de parámetros

Añade manejadores para los errores de parámetros más comunes:

```
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.web.bind.MissingServletRequestParameterException;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

// ...

@ExceptionHandler(HttpMessageNotReadableException.class)
public ResponseEntity<ErrorResponse> handleMensajeNoLeible(
    HttpMessageNotReadableException ex,
    HttpServletRequest request) {
    return buildErrorResponse(
        HttpStatus.BAD_REQUEST,
        "MENSAJE_NO_LEIBLE",
        "El cuerpo de la petición no se puede leer. Verifique el formato JSON",
        request,
        null
    );
}

@ExceptionHandler(MissingServletRequestParameterException.class)
public ResponseEntity<ErrorResponse> handleParametroFaltante(
    MissingServletRequestParameterException ex,
    HttpServletRequest request) {
    return buildErrorResponse(
        HttpStatus.BAD_REQUEST,
        "PARAMETRO_FALTANTE",
        "Falta el parámetro obligatorio: " + ex.getParameterName(),
        request,
        Map.of("parametro", ex.getParameterName())
    );
}

@ExceptionHandler(MethodArgumentTypeMismatchException.class)
public ResponseEntity<ErrorResponse> handleTipoIncorrecto(
    MethodArgumentTypeMismatchException ex,
    HttpServletRequest request) {
    return buildErrorResponse(
        HttpStatus.BAD_REQUEST,
        "TIPO_INCORRECTO",
        "El parámetro " + ex.getName() + " tiene un tipo incorrecto",
        request,
        Map.of("parametro", ex.getName(), "valor", ex.getValue())
    );
}
```

`HttpMessageNotReadableException` → 400 con código `MENSAJE_NO_LEIBLE`. `MissingServletRequestParameterException` → 400 con código `PARAMETRO_FALTANTE`. `MethodArgumentTypeMismatchException` → 400 con código `TIPO_INCORRECTO`.

Pregunta de reflexión: ¿Qué diferencia hay entre `PARAMETRO_FALTANTE` y `TIPO_INCORRECTO`?

Paso 7 - Arrancar y probar errores

Reinicia la aplicación y prueba varios tipos de error:

Recurso no encontrado

```
curl -i http://localhost:8080/api/v1/alumnos/99999
```

DNI duplicado

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  --data-binary @- <<'JSON'
{
  "nombre": "Ana",
  "apellidos": "García",
  "dni": "12345678A",
  "fechaNacimiento": "2010-05-12",
  "curso": "5º Primaria"
}
JSON
```

JSON mal formado

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre": "Ana",}'
```

Tipo de parámetro incorrecto

```
curl -i http://localhost:8080/api/v1/alumnos/abc
```

Pregunta de reflexión: ¿Qué estructura tienen las respuestas de error? ¿Todos los errores tienen los mismos campos?

Paso 8 - Verificar el error 404

Ejecuta:

```
curl -i http://localhost:8080/api/v1/alumnos/99999
```

Verás algo como:

```
{
  "timestamp": "2025-01-15T10:00:00Z",
  "status": 404,
  "codigo": "RECURSO_NO_ENCONTRADO",
  "mensaje": "No se encontró Alumno con id 99999",
  "path": "/api/v1/alumnos/99999",
  "detalles": {
    "recurso": "Alumno",
    "id": 99999
  }
}
```

El error incluye los campos básicos y los detalles específicos del recurso.

Pregunta de reflexión: ¿Qué campos son específicos de este tipo de error? ¿Qué campos son comunes a todos los errores?

Paso 9 - Verificar el error de validación

Ejecuta:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
--data-binary @- <<'JSON'
{
  "nombre": "",
  "apellidos": "López",
  "dni": "123",
  "fechaNacimiento": "2010-05-12",
  "curso": "4º"
}
JSON
```

Verás algo como:

```
{
  "timestamp": "2025-01-15T10:00:00Z",
  "status": 400,
  "codigo": "VALIDACION",
  "mensaje": "La petición tiene errores de validación",
  "path": "/api/v1/alumnos",
  "errors": [
    {
      "field": "nombre",
      "rejectedValue": "",
      "message": "El nombre es obligatorio"
    },
    {
      "field": "dni",
      "rejectedValue": "123",
      "message": "El DNI debe tener 9 caracteres"
    }
  ]
}
```

Pregunta de reflexión: ¿Qué diferencia hay entre el campo detalles y el campo errors?

Paso 10 - Escribir tests para las respuestas de error

Añade tests al AlumnoControllerTest:

```
@Test
void consultar_debeDevolver404ConEstructuraEstandar() throws Exception {
    when(service.consultar(999L)).thenReturn(Optional.empty());

    mockMvc.perform(get("/api/v1/alumnos/999"))
        .andExpect(status().isNotFound())
        .andExpect(jsonPath("$.timestamp").exists())
        .andExpect(jsonPath("$.status").value(404))
        .andExpect(jsonPath("$.codigo").value("RECURSO_NO_ENCONTRADO"))
        .andExpect(jsonPath("$.mensaje").exists())
        .andExpect(jsonPath("$.path").value("/api/v1/alumnos/999"))
        .andExpect(jsonPath("$.detalles.recurso").value("Alumno"))
        .andExpect(jsonPath("$.detalles.id").value(999));
}

@Test
void crear_debeDevolver400ConErroresDeValidacion() throws Exception {
    mockMvc.perform(post("/api/v1/alumnos"))
```

```

        .contentType(MediaType.APPLICATION_JSON)
        .content("""
{"nombre":"","apellidos":"López",
  "dni":"123", "fechaNacimiento":"2010-05-12",
  "curso":"4º"}
"""))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.codigo").value("VALIDACION"))
        .andExpect(jsonPath("$.errors").isArray())
        .andExpect(jsonPath("$.errors.length()").value(2));
    }

@Test
void crear_debeDevolver409ConEstructuraEstandar() throws Exception {
    when(service.crear(any())).thenThrow(
        new RecursoDuplicadoException("Alumno", "dni", "12345678A"));
    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
{"nombre":"Ana", "apellidos":"García",
  "dni":"12345678A", "fechaNacimiento":"2010-05-12",
  "curso":"5º Primaria"}
"""))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.codigo").value("RECURSO_DUPLICADO"))
        .andExpect(jsonPath("$.detalles.campo").value("dni"))
        .andExpect(jsonPath("$.detalles.valor").value("12345678A"));
}

```

Los tres tests verifican:

- La estructura estándar del error 404.
- Los errores de validación con el array errors.
- El error 409 con los detalles del duplicado.

Pregunta de reflexión: ¿Qué ventaja tiene verificar la estructura del error en lugar de solo el código de estado?

Paso 11 - Errores comunes del ejercicio

Error	Causa	Solución
Los campos del error no aparecen	Falta @JsonInclude(NON_NULL) o getters	Añadirlos
El timestamp es null	No se ha inicializado	Usar Instant.now().toString()
El path es incorrecto	Se construye mal desde la petición	Verificar request.getRequestURI()
El array errors está vacío	No se han mapeado los FieldError	Revisar el stream
detalles no aparece en el JSON	Es null y @JsonInclude(NON_NULL) lo omite	Es correcto si no hay detalles
El error técnico expone detalles	Se devuelve ex.getMessage()	Usar un mensaje genérico
Los tests fallan por el timestamp	Cambia en cada ejecución	Verificar existencia, no valor exacto

Error	Causa	Solución
El error 500 no se captura	Falta <code>@ExceptionHandler(Exception.class)</code>	Añadir el manejador genérico

Paso 12 - Reto resuelto — Añadir un campo `traceId` a las respuestas de error

Reto: Añadir un campo `traceId` a las respuestas de error. El `traceId` es un identificador único que permite al servidor localizar el error en los logs.

Solución paso a paso:

Subpaso 1. Añadir el campo a `ErrorResponse`:

```
private String traceId;

public String getTraceId() { return traceId; }
public void setTraceId(String traceId) { this.traceId = traceId; }
```

Subpaso 2. Generar el `traceId` en el método `buildErrorResponse`:

```
private ResponseEntity<ErrorResponse> buildErrorResponse(
    HttpStatus status,
    String codigo,
    String mensaje,
    HttpServletRequest request,
    Map<String, Object> detalles) {
    ErrorResponse error = new ErrorResponse();
    error.setTimestamp(Instant.now().toString());
    error.setStatus(status.value());
    error.setCodigo(codigo);
    error.setMensaje(mensaje);
    error.setPath(request.getRequestURI());
    error.setDetalles(detalles);
    error.setTraceId(UUID.randomUUID().toString());
    return ResponseEntity.status(status).body(error);
}
```

`UUID.randomUUID().toString()` genera un identificador único.

Subpaso 3. Registrar el `traceId` en los logs:

```
private static final Logger log =
    LoggerFactory.getLogger(GlobalExceptionHandler.class);

// En cada manejador, antes de construir la respuesta:
log.error("Error {}: {} - traceId: {}", codigo, mensaje, traceId);
```

Subpaso 4. Reiniciar y probar:

```
curl -i http://localhost:8080/api/v1/alumnos/99999
```

Verás el `traceId` en la respuesta:

```
{
  "timestamp": "2025-01-15T10:00:00Z",
  "status": 404,
  "codigo": "RECURSO_NO_ENCONTRADO",
  "mensaje": "No se encontró Alumno con id 99999",
  "path": "/api/v1/alumnos/99999",
  "traceId": "a1b2c3d4-e5f6-7890-1234-567890abcdef",
```

```

    "detalles": {
        "recurso": "Alumno",
        "id": 99999
    }
}

```

Pregunta de reflexión: ¿Qué ventaja tiene el traceId para el soporte técnico?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Un DTO ErrorResponse con: timestamp, status, codigo, mensaje, path, detalles, errors, traceId.
- Un DTO ValidationError con: field, rejectedValue, message.
- Un GlobalExceptionHandler que devuelve ResponseEntity en todos los manejadores.
- Manejadores para: RecursoNoEncontradoException, RecursoDuplicadoException, OperacionNoPermitidaException, ErrorTecnicoException, MethodArgumentNotValidException, HttpMessageNotReadableException, MissingServletRequestParameterException, MethodArgumentTypeMismatchException, Exception.
- Tests que verifican la estructura de las respuestas de error.

Resumen técnico

El ejercicio ha demostrado: - DTOs tipados: ErrorResponse y ValidationError. - Estructura estándar: campos básicos + detalles + errors + traceId. - @JsonInclude(NON_NULL): omite campos nulos. - buildErrorResponse: método de ayuda para no repetir código. - Manejadores: uno por tipo de excepción. - Errores de validación: array errors con field, rejectedValue, message. - Errores de parámetros: HttpMessageNotReadableException, MissingServletRequestParameterException, MethodArgumentTypeMismatchException. - traceId: identificador único para trazabilidad. - Tests: verificar código, campos y estructura.

Conclusión

En este punto 5.2 hemos creado respuestas de error estándar para la API:

- DTOs: ErrorResponse y ValidationError.
- Estructura: timestamp, status, codigo, mensaje, path, detalles, errors, traceId.
- @JsonInclude(NON_NULL): omite campos nulos.
- GlobalExceptionHandler: un manejador por tipo de excepción.
- Errores de validación: array errors.
- Errores de parámetros: HttpMessageNotReadableException, MissingServletRequestParameterException, MethodArgumentTypeMismatchException.
- traceId: para trazabilidad.
- Tests: verificar la estructura del error.

En la práctica, hemos refactorizado el GlobalExceptionHandler para usar DTOs tipados, hemos añadido manejadores para errores de parámetros, y hemos escrito tests que verifican la estructura de las respuestas.

La idea clave: una estructura de error estándar hace que el cliente pueda manejar todos los errores de la misma forma. Un solo manejador, un solo formato, un solo lugar donde buscar la información.

En el siguiente punto, 5.3 – Manejo global de excepciones, profundizaremos en @RestControllerAdvice: cómo funciona internamente, cómo se aplica a todos los controladores, cómo se ordenan los manejadores, y cómo se integra con el resto de la aplicación. Fin del Punto 5.2.

Práctica 5.3 - Manejo global de excepciones

Contexto del ejercicio: Vamos a organizar el manejo global de excepciones en varios `@RestControllerAdvice` con `@Order`, a aplicar uno de ellos solo a un paquete, y a escribir tests que verifiquen el orden y la aplicación selectiva.

Requisitos previos: Tener el proyecto mi-proyecto con el `GlobalExceptionHandler` del punto 5.2.

Paso 1 - Repasar el estado actual

Abre el `GlobalExceptionHandler` y observa su estructura. Actualmente es un solo `@RestControllerAdvice` con todos los manejadores. Vamos a dividirlo en tres.

Pregunta de reflexión: ¿Qué ventaja tiene dividir el manejador en varios según el tipo de error?

Paso 2 - Crear el `ValidationExceptionHandler`

Crea un `@RestControllerAdvice` específico para errores de validación:

```
package es.mecd.demo.miproyecto.common.exception;

import es.mecd.demo.miproyecto.common.dto.error.ErrorResponse;
import es.mecd.demo.miproyecto.common.dto.error.ValidationError;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.MissingServletRequestParameterException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

import java.time.Instant;
import java.util.List;
import java.util.Map;

@RestControllerAdvice
@Order(1)
public class ValidationExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidacion(
        MethodArgumentNotValidException ex,
        HttpServletRequest request) {
        List<ValidationError> errores = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .map(fe -> new ValidationError(
                fe.getField(),
                fe.getRejectedValue(),
                fe.getDefaultMessage()))
            .toList();

        ErrorResponse error = new ErrorResponse();
        error.setTimestamp(Instant.now().toString());
        error.setStatus(HttpStatus.BAD_REQUEST.value());
    }
}
```

```
        error.setCodigo("VALIDACION");
        error.setMensaje("La petición tiene errores de validación");
        error.setPath(request.getRequestURI());
        error.setErrorserrores);
        return ResponseEntity.badRequest().body(error);
    }

    @ExceptionHandler(HttpMessageNotReadableException.class)
    public ResponseEntity<ErrorResponse> handleMensajeNoLeible(
        HttpMessageNotReadableException ex,
        HttpServletRequest request) {
        return buildError(
            HttpStatus.BAD_REQUEST,
            "MENSAJE_NO_LEIBLE",
            "El cuerpo de la petición no se puede leer. Verifique el formato JSON",
            request,
            null
        );
    }

    @ExceptionHandler(MissingServletRequestParameterException.class)
    public ResponseEntity<ErrorResponse> handleParametroFaltante(
        MissingServletRequestParameterException ex,
        HttpServletRequest request) {
        return buildError(
            HttpStatus.BAD_REQUEST,
            "PARAMETRO_FALTANTE",
            "Falta el parámetro obligatorio: " + ex.getParameterName(),
            request,
            Map.of("parametro", ex.getParameterName())
        );
    }

    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public ResponseEntity<ErrorResponse> handleTipoIncorrecto(
        MethodArgumentTypeMismatchException ex,
        HttpServletRequest request) {
        return buildError(
            HttpStatus.BAD_REQUEST,
            "TIPO_INCORRECTO",
            "El parámetro " + ex.getName() + " tiene un tipo incorrecto",
            request,
            Map.of("parametro", ex.getName(), "valor", ex.getValue())
        );
    }

    private ResponseEntity<ErrorResponse> buildError(
        HttpStatus status, String codigo, String mensaje,
        HttpServletRequest request, Map<String, Object> detalles) {
        ErrorResponse error = new ErrorResponse();
        error.setTimestamp(Instant.now().toString());
        error.setStatus(status.value());
        error.setCodigo(codigo);
        error.setMensaje(mensaje);
        error.setPath(request.getRequestURI());
        error.setDetalles(detalles);
    }
}
```

```

        return ResponseEntity.status(status).body(error);
    }
}

```

@Order(1) hace que este manejador tenga prioridad sobre los demás. Si hay dos manejadores para la misma excepción, se usa el de

menor @Order.

Pregunta de reflexión: ¿Por qué el manejador de validación tiene @Order(1)? ¿Qué pasaría si tuviera @Order(3)?

Paso 3 - Crear el BusinessExceptionHandler

Crea un @RestControllerAdvice para las excepciones de negocio:

```

package es.mecd.demo.miproyecto.common.exception;

import es.mecd.demo.miproyecto.common.dto.error.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;
import java.util.Map;

@RestControllerAdvice
@Order(2)
public class BusinessExceptionHandler {

    @ExceptionHandler(RecursoNoEncontradoException.class)
    public ResponseEntity<ErrorResponse> handleNoEncontrado(
        RecursoNoEncontradoException ex,
        HttpServletRequest request) {
        return buildError(
            HttpStatus.NOT_FOUND,
            ex.getCodigo(),
            ex.getMessage(),
            request,
            Map.of("recurso", ex.getRecurso(), "id", ex.getId())
        );
    }

    @ExceptionHandler(RecursoDuplicadoException.class)
    public ResponseEntity<ErrorResponse> handleDuplicado(
        RecursoDuplicadoException ex,
        HttpServletRequest request) {
        return buildError(
            HttpStatus.CONFLICT,
            ex.getCodigo(),
            ex.getMessage(),
            request,
            Map.of(
                "recurso", ex.getRecurso(),
                "campo", ex.getCampo(),

```

```

        "valor", ex.getValor()
    )
};
}

@ExceptionHandler(OperacionNoPermitidaException.class)
public ResponseEntity<ErrorResponse> handleOperacionNoPermitida(
    OperacionNoPermitidaException ex,
    HttpServletRequest request) {
    return buildError(
        HttpStatus.CONFLICT,
        ex.getCodigo(),
        ex.getMessage(),
        request,
        null
    );
}

@ExceptionHandler(AplicacionException.class)
public ResponseEntity<ErrorResponse> handleAplicacion(
    AplicacionException ex,
    HttpServletRequest request) {
    return buildError(
        HttpStatus.BAD_REQUEST,
        ex.getCodigo(),
        ex.getMessage(),
        request,
        null
    );
}

private ResponseEntity<ErrorResponse> buildError(
    HttpStatus status, String codigo, String mensaje,
    HttpServletRequest request, Map<String, Object> detalles) {
    ErrorResponse error = new ErrorResponse();
    error.setTimestamp(Instant.now().toString());
    error.setStatus(status.value());
    error.setCodigo(codigo);
    error.setMensaje(mensaje);
    error.setPath(request.getRequestURI());
    error.setDetalles(detalles);
    return ResponseEntity.status(status).body(error);
}
}

```

@ExceptionHandler(AplicacionException.class) captura cualquier excepción de negocio que no tenga un manejador específico. Es

un manejador de respaldo dentro del grupo de negocio.

Pregunta de reflexión: ¿Qué excepciones captura el manejador de AplicacionException que no capturen los manejadores específicos?

Paso 4 - Crear el GenericExceptionHandler

Crea un @RestControllerAdvice para errores técnicos y genéricos:

```
package es.mecd.demo.miproyecto.common.exception;

import es.mecd.demo.miproyecto.common.dto.error.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.core.annotation.Order;
import org.springframework.dao.DataAccessException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;
import java.util.UUID;

@RestControllerAdvice
@Order(3)
public class GenericExceptionHandler {
    private static final Logger log =
        LoggerFactory.getLogger(GenericExceptionHandler.class);

    @ExceptionHandler(ErrorTechnicoException.class)
    public ResponseEntity<ErrorResponse> handleErrorTechnico(
        ErrorTechnicoException ex,
        HttpServletRequest request) {
        String traceId = UUID.randomUUID().toString();
        log.error(
            "Error técnico en {} - traceId: {}",
            request.getRequestURI(), traceId, ex);
        return buildError(
            HttpStatus.INTERNAL_SERVER_ERROR,
            ex.getCodigo(),
            "Error interno del servidor",
            request,
            traceId
        );
    }

    @ExceptionHandler(DataAccessException.class)
    public ResponseEntity<ErrorResponse> handleDataAccess(
        DataAccessException ex,
        HttpServletRequest request) {
        String traceId = UUID.randomUUID().toString();
        log.error("Error de acceso a datos en {} - traceId: {}",
            request.getRequestURI(), traceId, ex);
        return buildError(
            HttpStatus.INTERNAL_SERVER_ERROR,
            "ERROR_BASE_DATOS",
            "Error al acceder a los datos",
            request,
            traceId
        );
    }

    @ExceptionHandler(Exception.class)
```

```

public ResponseEntity<ErrorResponse> handleGenerico(
    Exception ex,
    HttpServletRequest request) {
    String traceId = UUID.randomUUID().toString();
    log.error("Error inesperado en {} - traceId: {}",
        request.getRequestURI(), traceId, ex);
    return buildError(
        HttpStatus.INTERNAL_SERVER_ERROR,
        "ERROR_INTERNO",
        "Error interno del servidor",
        request,
        traceId
    );
}

private ResponseEntity<ErrorResponse> buildError(
    HttpStatus status, String codigo, String mensaje,
    HttpServletRequest request, String traceId) {
    ErrorResponse error = new ErrorResponse();
    error.setTimestamp(Instant.now().toString());
    error.setStatus(status.value());
    error.setCodigo(codigo);
    error.setMensaje(mensaje);
    error.setPath(request.getRequestURI());
    error.setTraceId(traceId);
    return ResponseEntity.status(status).body(error);
}
}

```

Este manejador:

- Registra los errores técnicos en los logs con la traza de pila completa.
- Genera un traceId para que el cliente pueda reportar el error al soporte.
- Devuelve un mensaje genérico al cliente, sin exponer detalles técnicos.

Pregunta de reflexión: ¿Por qué el manejador genérico registra la excepción completa en los logs pero devuelve un mensaje genérico al cliente?

Paso 5 - Eliminar el GlobalExceptionHandler antiguo

Elimina la clase GlobalExceptionHandler del punto 5.2. Ahora tienes tres manejadores especializados. Spring los registrará todos y los aplicará en orden.

Pregunta de reflexión: ¿Qué pasaría si dejaras el GlobalExceptionHandler antiguo además de los tres nuevos? ¿Habría conflictos?

Paso 6 - Arrancar y probar el orden

Reinicia la aplicación y prueba varios tipos de error:

```

# Error de validación (debería manejarlo ValidationExceptionHandler, @Order(1))
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  --data-binary @- <<'JSON'
{
  "nombre": "",
  "apellidos": "López",
  "dni": "123",
  "fechaNacimiento": "2010-05-12",

```

```
"curso": "4º"
}
JSON
```

```
# Recurso no encontrado (debería manejarlo BusinessExceptionHandler, @Order(2))
curl -i http://localhost:8080/api/v1/alumnos/99999
```

```
# Error inesperado (debería manejarlo GenericExceptionHandler, @Order(3))
# Provocar un NullPointerException en algún endpoint
```

Pregunta de reflexión: ¿Cómo puedes verificar que cada manejador se está aplicando correctamente?

Paso 7 - Aplicar un manejador a un paquete concreto

Vamos a crear un manejador específico para el paquete alumno que añada información extra a los errores:

```
package es.mecd.demo.miproyecto.alumno;

import es.mecd.demo.miproyecto.common.dto.error.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;

@RestControllerAdvice(basePackages = "es.mecd.demo.miproyecto.alumno")
@Order(0)
public class AlumnoExceptionHandler {

    @ExceptionHandler(AlumnoEspecificoException.class)
    public ResponseEntity<ErrorResponse> handleAlumnoEspecifico(
        AlumnoEspecificoException ex,
        HttpServletRequest request) {
        ErrorResponse error = new ErrorResponse();
        error.setTimestamp(Instant.now().toString());
        error.setStatus(HttpStatus.BAD_REQUEST.value());
        error.setCodigo("ALUMNO_ESPECIFICO");
        error.setMensaje(ex.getMessage());
        error.setPath(request.getRequestURI());
        return ResponseEntity.badRequest().body(error);
    }
}
```

basePackages = "es.mecd.demo.miproyecto.alumno" restringe este manejador a los controladores de ese paquete. @Order(0) le da prioridad sobre los demás.

Pregunta de reflexión: ¿Qué pasa si un controlador del paquete alumno lanza una excepción que también está manejada en el BusinessExceptionHandler global?

Paso 8 - Escribir tests para el orden

Añade tests al AlumnoControllerExceptionTest:

```

@Test
void crear_debeDevolver400ConValidationHandler() throws Exception {
    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
        {"nombre":"","apellidos":"López","dni":"123",
        "fechaNacimiento":"2010-05-12","curso":"4º"}
        """))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.codigo").value("VALIDACION"))
        .andExpect(jsonPath("$.errors").isArray());
}

@Test
void consultar_debeDevolver404ConBusinessHandler() throws Exception {
    when(service.consultar(999L))
        .thenThrow(new RecursoNoEncontradoException("Alumno", 999L));
    mockMvc.perform(get("/api/v1/alumnos/999"))
        .andExpect(status().isNotFound())
        .andExpect(jsonPath("$.codigo").value("RECURSO_NO_ENCONTRADO"));
}

@Test
void consultar_debeDevolver500ConGenericHandler() throws Exception {
    when(service.consultar(anyLong()))
        .thenThrow(new RuntimeException("Error inesperado"));

    mockMvc.perform(get("/api/v1/alumnos/1"))
        .andExpect(status().isInternalServerError())
        .andExpect(jsonPath("$.codigo").value("ERROR_INTERNO"))
        .andExpect(jsonPath("$.mensaje").value("Error interno del servidor"));
}

```

Los tres tests verifican que cada manejador se aplica al tipo de error correspondiente.

Pregunta de reflexión: ¿Qué ventaja tiene tener tres manejadores separados en lugar de uno solo?

Paso 9 - Verificar el traceId

Prueba un error inesperado y observa el traceId en la respuesta y en los logs:

```
curl -i http://localhost:8080/api/v1/alumnos/1
```

Verás algo como:

```

{
  "timestamp": "2025-01-15T10:00:00Z",
  "status": 500,
  "codigo": "ERROR_INTERNO",
  "mensaje": "Error interno del servidor",
  "path": "/api/v1/alumnos/1",
  "traceId": "alb2c3d4-e5f6-7890-1234-567890abcdef"
}

```

En los logs de la aplicación verás el mismo traceId con la traza de pila completa.

Pregunta de reflexión: ¿Cómo puede el soporte técnico usar el traceId para localizar el error?

Paso 10 - Añadir un manejador para errores de tipo

Añade un manejador específico para `MethodArgumentTypeMismatchException` si no lo tienes ya:

```
@ExceptionHandler(MethodArgumentTypeMismatchException.class)
public ResponseEntity<ErrorResponse> handleTipoIncorrecto(
    MethodArgumentTypeMismatchException ex,
    HttpServletRequest request) {
    return buildError(
        HttpStatus.BAD_REQUEST,
        "TIPO_INCORRECTO",
        "El parámetro " + ex.getName() + " tiene un tipo incorrecto",
        request,
        Map.of("parametro", ex.getName(), "valor", ex.getValue())
    );
}
```

Prueba:

```
curl -i http://localhost:8080/api/v1/alumnos/abc
```

Verás un 400 con el código `TIPO_INCORRECTO`.

Pregunta de reflexión: ¿Qué diferencia hay entre `TIPO_INCORRECTO` y `PARAMETRO_FALTANTE`?

Paso 11 - Errores comunes del ejercicio

Error	Causa	Solución
Los manejadores no se aplican	Falta <code>@RestControllerAdvice</code>	Añadirlo
Dos manejadores capturan la misma excepción	No hay <code>@Order</code>	Definir el orden
El manejador local captura antes	La prioridad local es mayor	Quitar el local o cambiar la lógica
El <code>traceId</code> no aparece	No se ha asignado	Asignarlo al construir la respuesta
El log no muestra la excepción	Falta pasar <code>ex</code> a <code>log.error</code>	Añadir el parámetro
Los tests fallan por el timestamp	El timestamp cambia	Usar <code>exists()</code> en vez de un valor exacto
El manejador de paquete no se aplica	<code>basePackages</code> está mal escrito	Verificar el paquete
Un error 404 termina como 500	El handler específico no gana	Verificar tipo y orden

Paso 12 - Reto resuelto — Manejador específico para ficheros

Reto: Añadir un manejador específico para el paquete `fichero` que maneje `FileNotFoundException` y devuelva un 404 con un código específico.

Solución paso a paso:

Subpaso 1. Crear el paquete `fichero` con un controlador y una excepción específica:

```
package es.mecd.demo.miproyecto.fichero;
public class FicheroNoEncontradoException extends RuntimeException {
    public FicheroNoEncontradoException(String mensaje) {
        super(mensaje);
    }
}
```

Subpaso 2. Crear el manejador específico:

```
package es.mecd.demo.miproyecto.fichero;

import es.mecd.demo.miproyecto.common.dto.error.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;

@RestControllerAdvice(basePackages = "es.mecd.demo.miproyecto.fichero")
@Order(0)
public class FicheroExceptionHandler {

    @ExceptionHandler(FicheroNoEncontradoException.class)
    public ResponseEntity<ErrorResponse> handleFicheroNoEncontrado(
        FicheroNoEncontradoException ex,
        HttpServletRequest request) {
        ErrorResponse error = new ErrorResponse();
        error.setTimestamp(Instant.now().toString());
        error.setStatus(HttpStatus.NOT_FOUND.value());
        error.setCodigo("FICHERO_NO_ENCONTRADO");
        error.setMensaje(ex.getMessage());
        error.setPath(request.getRequestURI());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }
}
```

Subpaso 3. Crear un controlador en el paquete fichero:

```
@RestController
@RequestMapping("/api/v1/ficheros")
public class FicheroController {

    @GetMapping("/{nombre}")
    public ResponseEntity<String> obtener(@PathVariable String nombre) {
        throw new FicheroNoEncontradoException(
            "No se encontró el fichero " + nombre);
    }
}
```

Subpaso 4. Probar:

```
curl -i http://localhost:8080/api/v1/ficheros/documento.pdf
```

Verás un 404 con el código FICHERO_NO_ENCONTRADO. El manejador específico del paquete fichero ha capturado la excepción.

Pregunta de reflexión: ¿Qué pasaría si el mismo controlador lanzara una RecursoNoEncontradoException? ¿Qué manejador la capturaría?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Tres `@RestControllerAdvice` globales: `ValidationExceptionHandler`, `BusinessExceptionHandler`, `GenericExceptionHandler`, con `@Order` 1, 2 y 3.
- Un `@RestControllerAdvice` específico para el paquete alumno (opcional).
- Un `@RestControllerAdvice` específico para el paquete fichero (opcional).
- Tests que verifican el orden y la aplicación selectiva.
- Un `traceId` en las respuestas de error técnico.
- Logs de errores con la traza completa. **## Resumen técnico**

El ejercicio ha demostrado:

- `@RestControllerAdvice`: manejo global de excepciones.
- `@Order`: orden de prioridad de los manejadores.
- `basePackages`: aplicación selectiva a un paquete.
- Jerarquía de excepciones: Spring usa el más específico.
- Manejador genérico: para excepciones no controladas.
- Logs: registrar errores técnicos con traza.
- `traceId`: para trazabilidad.
- Tests: verificar el orden y la estructura de los errores.

Conclusión

En este punto 5.3 hemos profundizado en el manejo global de excepciones: - `@RestControllerAdvice`: manejo global para todos los controladores. - Orden de búsqueda: local → global aplicable → global → por defecto. - `@Order`: ordena los manejadores globales. - Aplicación selectiva: por paquete, clase o anotación. - Excepciones de validación, seguridad y técnicas. - Buenas prácticas: un manejador por excepción, manejador genérico, logs, `traceId`. - Tests: verificar el código, la estructura y el orden.

En la práctica, hemos dividido el `GlobalExceptionHandler` en tres manejadores especializados, hemos aplicado uno a un paquete concreto, y hemos añadido `traceId` y logs.

La idea clave: centralizar el manejo de excepciones en un `@RestControllerAdvice` hace que la API sea predecible. Todos los errores tienen la misma estructura, y el cliente escribe un solo manejador.

En el siguiente punto, 5.4 - Configuración de CORS para consumo desde front-end, profundizaremos en CORS: qué es, por qué el navegador bloquea las peticiones entre orígenes, cómo se configura en Spring Boot, y cómo se integra con el resto de la aplicación.

Práctica 5.4 - Configuración básica de perfiles y properties

Contexto del ejercicio: Vamos a organizar la configuración del proyecto en tres perfiles: dev, test y prod. Externalizaremos la configuración de la base de datos y de la aplicación en `@ConfigurationProperties`. Veremos cómo se comporta la aplicación con cada perfil.

Requisitos previos: Tener el proyecto mi-proyecto con la configuración actual del Módulo 4.

Paso 1 - Repasar el estado actual

Abre `application.properties` y observa su contenido. Probablemente tiene toda la configuración mezclada: base de datos, propiedades personalizadas, logging. Vamos a organizarlo.

Pregunta de reflexión: ¿Qué propiedades crees que son comunes a todos los entornos y cuáles cambian?

Paso 2 - Reorganizar `application.properties`

Deja en `application.properties` solo las propiedades comunes:

```
spring.application.name=mi-proyecto
app.version=1.0.0
app.nombre-oficina=Oficina de Becas
```

```
# JPA común
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
```

spring.application.name: nombre lógico de la aplicación. Común a todos los entornos.

app.version y app.nombre-oficina: propiedades personalizadas. La versión es común; el nombre puede cambiar (lo sobrescribiremos en dev).

spring.jpa.hibernate.ddl-auto=update: en desarrollo y producción, update. En test, create-drop (lo pondremos en el perfil de test).

Nota sobre spring.jpa.open-in-view. El “Open Session in View” es un mecanismo de Spring Boot que mantiene abierta la sesión de Hibernate durante toda la petición HTTP, hasta que se envía la respuesta. Eso permite acceder a relaciones LAZY desde el controlador sin lanzar LazyInitializationException. Pero es una mala práctica: alarga la conexión a la base de datos, dificulta el control de transacciones y oculta problemas de diseño. Se recomienda desactivarlo (false) y transformar las entidades a DTOs dentro de la transacción, en el servicio. Spring Boot lo tiene activado por defecto (true) por razones históricas, pero conviene desactivarlo explícitamente.

Pregunta de reflexión: ¿Por qué spring.jpa.open-in-view=false es una buena práctica?

Paso 3 - Crear application-dev.properties

Crea src/main/resources/application-dev.properties:

```
# Servidor
server.port=8080

# Base de datos H2 en memoria
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.h2.console.enabled=true

# Logging
logging.level.es.mecd.demo=DEBUG
logging.level.org.springframework.web=DEBUG

# Propiedades personalizadas
app.nombre-oficina=Oficina de Becas (Desarrollo)
```

Pregunta de reflexión: ¿Por qué en desarrollo se usa create-drop y H2 en memoria?

Paso 4 - Crear application-test.properties

Crea src/main/resources/application-test.properties:

```
# Base de datos H2 en memoria para tests
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
```

```
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=false
spring.h2.console.enabled=false

# Logging mínimo
logging.level.es.mecd.demo=INFO
logging.level.org.springframework=WARN

# Propiedades personalizadas
app.nombre-oficina=Oficina de Becas (Test)

create-drop en tests garantiza que la base de datos se crea limpia en cada ejecución. spring.jpa.show-sql=false para no llenar los logs de tests.

logging.level.org.springframework=WARN para reducir el ruido.
```

Pregunta de reflexión: ¿Por qué en test se usa create-drop en lugar de update?

Paso 5 - Crear application-prod.properties

Crea src/main/resources/application-prod.properties:

```
# Servidor
server.port=8443

# Base de datos PostgreSQL
spring.datasource.url=jdbc:postgresql://${DB_HOST}:${DB_PORT:5432}/${DB_NAME}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false

# Logging
logging.level.es.mecd.demo=INFO
logging.level.org.springframework=WARN
logging.file.name=/var/log/mi-proyecto/app.log

# Propiedades personalizadas
app.nombre-oficina=Oficina de Becas

${DB_HOST}, ${DB_USER}, ${DB_PASSWORD}: variables de entorno. No se escriben en el archivo.

${DB_PORT:5432}: variable de entorno con valor por defecto. Si DB_PORT no está definida, se usa 5432.

spring.jpa.show-sql=false: en producción no se muestra el SQL.
logging.file.name: los logs se escriben en un archivo además de la consola.
```

Pregunta de reflexión: ¿Por qué las credenciales de la base de datos se leen de variables de entorno?

Paso 6 - Activar el perfil dev

Añade a application.properties:

```
spring.profiles.active=dev
```

Reinicia la aplicación. En los logs verás:

The following 1 profile is active: "dev"

Y las propiedades de application-dev.properties se aplicarán. El puerto es 8080, la base de datos es H2 en memoria, y el logging está en DEBUG.

Pregunta de reflexión: ¿Cómo puedes verificar qué perfil está activo?

Paso 7 - Probar con el perfil prod

Para probar el perfil prod, cambia spring.profiles.active a prod o pásalo por línea de comandos:

```
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar --spring.profiles.active=prod
```

Verás que la aplicación intenta conectarse a PostgreSQL. Si no tienes PostgreSQL instalado, fallará. Para probarlo sin PostgreSQL, podemos cambiar temporalmente application-prod.properties para que use H2:

```
spring.datasource.url=jdbc:h2:mem:proddb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
```

Restauración obligatoria: esta sustitución por H2 es sólo una prueba. Antes de cerrar el paso, restaura application-prod.properties con la configuración PostgreSQL original y deja spring.profiles.active=dev en application.properties. Ejecuta después ./mvnw test para comprobar que el snapshot final ha vuelto al estado esperado.

Pregunta de reflexión: ¿Qué pasaría si DB_USER no estuviera definida como variable de entorno?

Paso 8 - Crear la clase AppProperties

Vamos a agrupar las propiedades personalizadas en una clase @ConfigurationProperties. Crea AppProperties en el paquete config:

```
package es.mecd.demo.miproyecto.config;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;
import org.springframework.validation.annotation.Validated;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;

@Component
@ConfigurationProperties(prefix = "app")
@Validated
public class AppProperties {

    @NotBlank
    private String nombreOficina;

    @NotBlank
    @Pattern(regexp = "\\d+\\.\\d+\\.\\d+")
    private String version;

    public String getNombreOficina() { return nombreOficina; }
    public void setNombreOficina(String nombreOficina) {
        this.nombreOficina = nombreOficina;
    }
}
```

```

    public String getVersion() { return version; }
    public void setVersion(String version) { this.version = version; }
}

```

@ConfigurationProperties(prefix = "app") mapea todas las propiedades que empiezan por app..
 @Validated activa la validación. @NotBlank y @Pattern validan que las propiedades estén presentes y con el formato correcto.

Pregunta de reflexión: ¿Qué pasa si app.version no está definida en ningún archivo de propiedades?

Paso 9 - Añadir un bean condicional por perfil

Vamos a añadir un CommandLineRunner que solo se ejecute en desarrollo. Modifica DatosInicialesConfig:

```

package es.mecd.demo.miproyecto.config;
import es.mecd.demo.miproyecto.alumno.Alumno;
import es.mecd.demo.miproyecto.alumno.AlumnoRepository;
import es.mecd.demo.miproyecto.curso.Curso;
import es.mecd.demo.miproyecto.curso.CursoRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

import java.time.LocalDate;

@Configuration
public class DatosInicialesConfig {

    @Bean
    @Profile("dev")
    public CommandLineRunner cargarDatosDev(AlumnoRepository alumnoRepository,
        CursoRepository cursoRepository) {
        return args -> {
            if (cursoRepository.count() == 0) {
                Curso curso = cursoRepository.save(new Curso("5º Primaria"));
                Curso curso2 = cursoRepository.save(new Curso("6º Primaria"));

                Alumno ana = new Alumno("Ana", "García", "12345678A",
                    LocalDate.of(2010, 5, 12));
                ana.setCurso(curso);
                alumnoRepository.save(ana);

                Alumno luis = new Alumno("Luis", "Pérez", "87654321B",
                    LocalDate.of(2009, 9, 3));
                luis.setCurso(curso2);
                alumnoRepository.save(luis);

                System.out.println("Datos de ejemplo cargados en desarrollo");
            }
        };
    }

    @Bean
    @Profile("prod")

```

```

public CommandLineRunner verificarDatosProd(AlumnoRepository alumnoRepository) {
    return args -> {
        if (alumnoRepository.count() == 0) {
            System.out.println(
                "ADVERTENCIA: No hay alumnos en la base de datos "
                + "de producción");
        } else {
            System.out.println(
                "Producción: " + alumnoRepository.count()
                + " alumnos cargados");
        }
    };
}
}
}

```

@Profile("dev") hace que el primer bean solo se registre en desarrollo. Carga datos de ejemplo.
 @Profile("prod") hace que el segundo solo se registre en producción. Verifica que hay datos.

Pregunta de reflexión: ¿Qué pasaría si un bean con @Profile("dev") fuera inyectado por otro bean que no tiene perfil?

Paso 10 - Arrancar con distintos perfiles

Arranca la aplicación con el perfil dev:

```
./mvnw spring-boot:run
```

Verás en los logs:

```
The following 1 profile is active: "dev" ... Datos de ejemplo cargados en desarrollo
```

Arranca con el perfil test:

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=test
```

Verás:

```
The following 1 profile is active: "test"
```

Y no se cargarán los datos de ejemplo (porque el bean tiene @Profile("dev")).

Arranca con el perfil prod:

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=prod
```

Verás:

```
The following 1 profile is active: "prod" ... Producción: 0 alumnos cargados
```

Pregunta de reflexión: ¿Qué diferencia hay entre los tres arranques?

Paso 11 - Errores comunes del ejercicio

Error	Causa	Solución
spring.profiles.active mal escrito	El perfil no se activa	Verificar el nombre
La propiedad no se lee	El perfil no está activo	Activar el perfil correcto
Could not resolve placeholder	Falta una variable de entorno	Definirla o dar valor por defecto
El bean no se registra	Falta @Profile	Añadirlo
NoSuchBeanDefinitionException	El bean es de un perfil no activo	Verificar el perfil

Error	Causa	Solución
La validación falla al arrancar	Falta una propiedad obligatoria	Definirla
Las propiedades del perfil no sobrescriben	application-{perfil}.properties está mal nombrado	Corregir el nombre
Los secretos aparecen en el repositorio	Están escritos en el archivo	Moverlos a variables de entorno
Se usa el perfil por defecto sin querer	No se ha activado ninguno	Activar el perfil correcto
El test usa configuración de producción	Falta el perfil de test	Añadir <code>@ActiveProfiles("test")</code>

Paso 12 - Reto resuelto — Test con perfil específico

Reto: Escribir un test que use el perfil test para verificar que la aplicación arranca correctamente con esa configuración.

Solución paso a paso:

Subpaso 1. Crear el test:

```
package es.mecd.demo.miproyecto;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;

@SpringBootTest
@ActiveProfiles("test")
class MiProyectoApplicationTest {

    @Test
    void contextLoads() {
        // Verifica que el contexto arranca con el perfil test
    }
}
```

`@ActiveProfiles("test")` activa el perfil test para este test. Spring Boot cargará `application.properties` + `application-test.properties`.

Subpaso 2. Ejecutar el test:

```
./mvnw test -Dtest=MiProyectoApplicationTest
```

Si todo va bien, el test pasa. Si hay un problema de configuración en el perfil test, el test falla con un mensaje claro.

Pregunta de reflexión: ¿Qué ventaja tiene tener un test que verifica que el contexto arranca con el perfil de test?

Resultado esperado global

Al final del ejercicio, deberías tener:

- `application.properties` con propiedades comunes.
- `application-dev.properties`, `application-test.properties`, `application-prod.properties` con propiedades específicas.
- Una clase `AppProperties` con `@ConfigurationProperties` y validación.
- Beans condicionales por perfil (`CommandLineRunner` para dev y prod).
- Un test que verifica el perfil test.

Resumen técnico

El ejercicio ha demostrado:

- `application.properties`: configuración común.
- `application-{perfil}.properties`: configuración por entorno.
- `spring.profiles.active`: activa el perfil.
- `@Profile`: beans condicionales.
- `@ConfigurationProperties`: agrupa propiedades en una clase.
- `@Validated`: valida las propiedades al arrancar.
- Variables de entorno: para secretos.
- `spring.jpa.open-in-view`: mala práctica, desactivar.
- Perfiles en tests: `@ActiveProfiles("test")`. ## Conclusión

En este punto 5.4 hemos profundizado en la configuración con perfiles y properties:

- `application.properties`: configuración común.
- `application.yml`: alternativa YAML.
- Perfiles: dev, test, prod.
- `application-{perfil}.properties`: configuración específica.
- `spring.profiles.active`: activa el perfil.
- `@Profile`: beans condicionales.
- `@ConfigurationProperties`: agrupa propiedades.
- `@Validated`: valida las propiedades.
- Prioridad: argumentos > variables de entorno > perfil > base.
- Secretos: variables de entorno.

En la práctica, hemos organizado la configuración del proyecto en tres perfiles, hemos externalizado las propiedades personalizadas en `AppProperties`, hemos usado `@Profile` para beans condicionales, y hemos probado la aplicación con cada perfil.

La idea clave: la configuración debe ser externalizable y adaptable a cada entorno. Los perfiles permiten tener la misma aplicación con configuraciones distintas sin recompilar.

En el siguiente punto, 5.5 – Configuración de CORS, veremos cómo permitir que un front-end pueda consumir la API desde otro origen. Fin del Punto 5.4.

Práctica 5.5 - Configuración de CORS para consumo desde front-end

Contexto del ejercicio: Vamos a configurar CORS en el proyecto `mi-proyecto` para permitir peticiones desde un front-end en `http://localhost:3000`. Veremos cómo se comporta con curl y con el navegador, y cómo diagnosticar errores.

Requisitos previos: Tener el proyecto `mi-proyecto` con la configuración por perfiles del punto 5.4.

Paso 1 - Repasar el estado actual

Arranca la aplicación y prueba un endpoint con curl:

```
curl http://localhost:8080/api/v1/alumnos
```

Verás la lista de alumnos. No hay problema de CORS porque curl no aplica la política de mismo origen. El problema solo aparece en el navegador.

Pregunta de reflexión: ¿Por qué curl no tiene problemas de CORS?

Paso 2 - Simular una petición desde otro origen

Vamos a simular una petición desde otro origen con curl. Enviamos la cabecera Origin como la enviaría un navegador:

```
curl -i http://localhost:8080/api/v1/alumnos \
-H "Origin: http://localhost:3000"
```

Observa la respuesta. Si no ves ninguna cabecera Access-Control-Allow-Origin, el navegador bloquearía esta respuesta.

Pregunta de reflexión: ¿Qué cabecera debería incluir la respuesta para que el navegador la acepte?

Paso 3 - Crear la clase CorsConfig

Vamos a configurar CORS globalmente. Crea la clase CorsConfig en el paquete config:

```
package es.mecd.demo.miproyecto.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
            .allowedHeaders("*")
            .exposedHeaders("Location")
            .allowCredentials(true)
            .maxAge(3600);
    }
}
```

`addMapping("/api/**")` aplica la configuración a todos los endpoints bajo `/api/`.

`allowedOrigins("http://localhost:3000")` permite solo ese origen.

`allowedMethods(...)` incluye `OPTIONS`, que es fundamental para el preflight.

`allowedHeaders("*")` permite cualquier cabecera.

`exposedHeaders("Location")` permite al front-end leer la cabecera `Location` de un `POST`.

`allowCredentials(true)` permite cookies y cabecera `Authorization`.

`maxAge(3600)` cachea el preflight durante 1 hora.

Pregunta de reflexión: ¿Por qué `allowedMethods` debe incluir `OPTIONS`?

Paso 4 - Arrancar y probar con curl

Reinicia la aplicación y prueba de nuevo con la cabecera Origin:

```
curl -i http://localhost:8080/api/v1/alumnos \
-H "Origin: http://localhost:3000"
```

Ahora verás en la respuesta:

```
HTTP/1.1 200
Access-Control-Allow-Origin: http://localhost:3000
Access-Control-Allow-Credentials: true
```

...

La cabecera Access-Control-Allow-Origin indica al navegador que ese origen está permitido.

Pregunta de reflexión: ¿Qué pasaría si el origen de la petición fuera http://localhost:4200?

Paso 5 - Probar con un origen no permitido

Prueba con un origen distinto:

```
curl -i http://localhost:8080/api/v1/alumnos \
-H "Origin: http://localhost:4200"
```

Verás que la respuesta no incluye la cabecera Access-Control-Allow-Origin (o incluye un valor distinto). El navegador bloquearía esta respuesta.

Pregunta de reflexión: ¿Qué habría que hacer para permitir http://localhost:4200 también?

Paso 6 - Simular un preflight

Vamos a simular un preflight OPTIONS:

```
curl -i -X OPTIONS http://localhost:8080/api/v1/alumnos \
-H "Origin: http://localhost:3000" \
-H "Access-Control-Request-Method: POST" \
-H "Access-Control-Request-Headers: content-type,authorization"
```

Verás algo como:

```
HTTP/1.1 200
Access-Control-Allow-Origin: http://localhost:3000
Access-Control-Allow-Methods: GET,POST,PUT,PATCH,DELETE,OPTIONS
Access-Control-Allow-Headers: content-type,authorization
Access-Control-Max-Age: 3600
Access-Control-Allow-Credentials: true
```

El servidor responde al preflight con las cabeceras que el navegador necesita.

Pregunta de reflexión: ¿Qué métodos y cabeceras indica el servidor que están permitidos?

Paso 7 - Probar con un método no permitido

Prueba un preflight para un método no permitido:

```
curl -i -X OPTIONS http://localhost:8080/api/v1/alumnos \
-H "Origin: http://localhost:3000" \
-H "Access-Control-Request-Method: TRACE"
```

El servidor responderá sin incluir TRACE en Access-Control-Allow-Methods. El navegador bloquearía la petición real.

Pregunta de reflexión: ¿Qué pasaría si el front-end intentara usar TRACE sin que esté permitido?

Paso 8 - Configurar CORS con perfiles

Vamos a externalizar los orígenes a application.properties. Modifica CorsConfig:

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Value("${cors.allowed-origins}")
    private String[] allowedOrigins;

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins(allowedOrigins)
            .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
            .allowedHeaders("*")
            .exposedHeaders("Location")
            .allowCredentials(true)
            .maxAge(3600);
    }
}
```

Y añade la propiedad a application-dev.properties:

```
cors.allowed-origins=http://localhost:3000,http://localhost:4200
```

Y a application-prod.properties:

```
cors.allowed-origins=https://sede.educacion.gob.es
```

Ahora la lista de orígenes cambia según el perfil activo. En desarrollo se permiten varios orígenes; en producción, solo el oficial.

Pregunta de reflexión: ¿Qué ventaja tiene externalizar los orígenes a application-dev.properties?

Paso 9 - Manejar CORS en errores

Vamos a verificar que CORS también se aplica a las respuestas de error. Prueba con un origen permitido:

```
curl -i http://localhost:8080/api/v1/alumnos/99999 \
-H "Origin: http://localhost:3000"
```

Verás el error 404 con las cabeceras CORS:

```
HTTP/1.1 404
Access-Control-Allow-Origin: http://localhost:3000
Content-Type: application/json
```

```
{
  "timestamp": "...",
  "status": 404,
  "codigo": "RECURSO_NO_ENCONTRADO",
  ... }
```

El front-end puede leer el error porque las cabeceras CORS están presentes.

Pregunta de reflexión: ¿Qué pasaría si el servidor devolviera un 500 sin cabeceras CORS?

Paso 10 - Errores comunes del ejercicio

Error	Causa	Solución
El navegador bloquea todas las peticiones	Falta Access-Control-Allow-Origin	Configurar CORS
El preflight falla	OPTIONS no está permitido	Añadirlo a allowedMethods
Error con credenciales	allowedOrigins("*") junto a allowCredentials(true)	Especificar orígenes concretos
El front-end no puede leer Location	Falta exposedHeaders("Location")	Añadirlo
CORS funciona con curl pero no con el navegador	curl no aplica Same-Origin Policy	Probar desde el navegador
Las cookies no se envían	Falta allowCredentials(true)	Añadirlo si procede
El preflight se envía en cada petición	Falta maxAge	Configurarlo
Spring Security bloquea OPTIONS	Falta integrar CORS con Security	Configurarlo en el Módulo 6
El error 500 no tiene cabeceras CORS	El flujo de error evita la configuración MVC	Revisar el handler y CORS

Paso 11 - Reto resuelto — Probar CORS con un front-end real

Reto: Crear un front-end mínimo con HTML y JavaScript que haga una petición al back-end y verificar que CORS funciona.

Solución paso a paso:

Subpaso 1. Crear un archivo index.html en el escritorio (o en cualquier carpeta):

```
<!DOCTYPE html>
<html>
<head>
  <title>Prueba CORS</title>
</head>
<body>
  <h1>Prueba de CORS</h1>
  <button onclick="consultarAlumnos()">Consultar alumnos</button>
  <button onclick="crearAlumno()">Crear alumno</button>
  <pre id="resultado"></pre>

  <script>
    async function consultarAlumnos() {
      try {
        const response = await fetch(
          'http://localhost:8080/api/v1/alumnos');
        const data = await response.json();
        document.getElementById('resultado').textContent =
          JSON.stringify(data, null, 2);
      } catch (error) {
        document.getElementById('resultado').textContent =
          'Error: ' + error.message;
      }
    }

    async function crearAlumno() {
      try {
```

```

    const response = await fetch(
      'http://localhost:8080/api/v1/alumnos', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({
          nombre: 'Ana',
          apellidos: 'García',
          dni: '99999999Z',
          fechaNacimiento: '2010-05-12',
          curso: '5º Primaria'
        })
      });
    const data = await response.json();
    document.getElementById('resultado').textContent =
      JSON.stringify(data, null, 2);
  } catch (error) {
    document.getElementById('resultado').textContent =
      'Error: ' + error.message;
  }
}
</script>
</body>
</html>

```

Subpaso 2. Abrir el archivo desde un servidor local. Como no quieres instalar Node, puedes abrir el archivo directamente con `file://`,

pero el origen sería `null`. Para que sea un origen real, abre un servidor simple con Python:

```
python3 -m http.server 3000
```

Ahora el archivo está en `http://localhost:3000/index.html`.

Subpaso 3. Abrir `http://localhost:3000` en el navegador. Pulsar “Consultar alumnos” y “Crear alumno”.

Verás las respuestas del back-end. El navegador ha enviado las peticiones y ha aceptado las respuestas porque CORS está configurado. **Subpaso 4.** Ahora para el servidor Python (Ctrl+C), cambia la configuración de CORS para que solo permita `http://localhost:4000`, y

vuelve a arrancar el servidor en el puerto 3000. Recarga la página y prueba de nuevo. El navegador bloqueará la petición y mostrarás el error en el pre.

Pregunta de reflexión: ¿Qué diferencia hay entre abrir el archivo con `file://` y abrirlo desde un servidor en `http://localhost:3000`?

Restauración: después de la prueba de bloqueo, deja `cors.allowed-origins` con los orígenes `http://localhost:3000` y `http://localhost:4200` en el perfil dev.

Resultado esperado global

Al final del ejercicio, deberías tener:

- Una clase `CorsConfig` que configura CORS globalmente.
- CORS configurado con orígenes externalizados a `application-dev.properties` y `application-prod.properties`.
- CORS verificado con curl (petición simple, preflight, método no permitido).
- (Opcional) CORS verificado con un front-end real.

Resumen técnico

El ejercicio ha demostrado:

- Same-Origin Policy: los navegadores bloquean peticiones entre orígenes distintos.
- CORS: mecanismo que permite al servidor autorizar al navegador.
- Peticiones simples vs preflight: las que no cumplen condiciones simples envían OPTIONS primero.
- WebMvcConfigurer: configuración global.
- @CrossOrigin: configuración por controlador.
- Cabeceras CORS: Access-Control-Allow-Origin, Access-Control-Allow-Methods, Access-Control-Allow-Headers, Access-Control-Max-Age, Access-Control-Allow-Credentials, Access-Control-Expose-Headers.
- Perfiles: orígenes distintos según entorno.
- Errores comunes: OPTIONS, * con credenciales, Location, Spring Security.

Conclusión

En este punto 5.5 hemos profundizado en CORS:

- Same-Origin Policy: los navegadores bloquean peticiones entre orígenes distintos.
- CORS: mecanismo de autorización del servidor al navegador.
- Peticiones simples vs preflight: OPTIONS antes de la petición real.
- Configuración global: WebMvcConfigurer.
- Configuración por controlador: @CrossOrigin.
- Configuración con Spring Security: se hará en el Módulo 6.
- Perfiles: orígenes distintos según entorno.
- Errores comunes: OPTIONS, credenciales, Location, seguridad.

En la práctica, hemos configurado CORS en el proyecto, hemos probado peticiones simples y preflight con curl, hemos externalizado los orígenes a los perfiles, y hemos verificado que los errores también llevan cabeceras CORS.